基于线性回归的共享单车租赁需求预测

温度、湿度、风速、降雨量、降雪量、季节与节假日特征下的
需求建模 ・ 数据处理 ・ 三种求解交叉验证 ・ 训练/测试泛化分析

姓　　名： _____

学　　号： _____

提交日期： 2026 年 10 月（作业发布后两周内）

数据来源： UCI Seoul Bike Sharing Demand（作业提供子集：训练 100 条 / 测试 60 条）

交付形式： PDF 电子报告 + 可复现代码、图件与结果文件

摘要

本文以温度、湿度、风速、降雨量、降雪量与季节、节假日共 9 个派生特征建立多元线性回归模型（$y = b + \Sigma w_i x_i$），对共享单车租赁需求进行预测：先对类别特征做哑变量编码、对连续特征做标准化（仅用训练集统计量），再分别用正规方程、scikit-learn 与批量梯度下降三种方法独立求解，三者参数最大偏差仅 9.3e-13；模型在训练集上 $R^2 = 0.506$、RMSE = 471.0，测试集上 $R^2 = 0.345$、RMSE = 508.2，相对均值基线（RMSE = 631.2）降低误差 19.5%。残差诊断揭示线性假设对高需求样本存在系统性低估（预测值标准差仅为真实值的 71.1%），补充实验表明引入温度二次项与“温度×季节”交互项可把测试集 R^2 提升至 0.461。

关键词：线性回归；最小二乘；正规方程；梯度下降；特征标准化；共享单车需求预测

目　录

任务完成情况自查表 3

一、任务理解与数据概况 4

1.1 问题定义 4

1.2 数据集概况与质量检查 4

1.3 探索性分析：特征与需求的关系 5

二、数据处理与特征工程 8

2.1 类别特征编码：哑变量与基准类别 8

2.2 连续特征标准化（Z-score） 8

三、线性回归模型与求解过程 10

3.1 模型形式与设计矩阵 10

3.2 最小二乘的闭式解：正规方程 10

3.3 三种求解方法的交叉验证 10

3.4 参数反变换：回到原始物理尺度 11

3.5 模型学习结果：参数与显著性检验 11

3.6 共线性诊断（VIF） 13

四、模型在训练集与测试集上的表现 14

4.1 评价指标 14

4.2 训练集与测试集结果总览 14

4.3 残差诊断：线性假设是否成立? 15

4.4 预测值的分布结构与分段误差 16

4.5 结果小结与局限性 16

五、改进探索与讨论 17

六、结论 18

附录 A 完整代码（可复现） 19

附录 B 交付文件清单与复现说明 25

任务完成情况自查表

下表逐条对照作业的五项要求，标注本报告中的完成位置，便于评阅。

#	作业要求	完成情况	对应位置
1	建立线性回归模型 $y = b + \Sigma w_i x_i$	完成。§3.1 给出模型形式；§2 给出 9 维特征的设计矩阵构造（含哑变量与标准化）	§2、§3.1
2	用 100 条训练数据训练模型，说明数据处理与求解过程，给出参数 b 与各 w_i	完成。§2 数据处理与特征工程；§3.2–3.4 最小二乘推导 + 正规方程/梯度下降两种实现互验；§3.5 给出全部参数（表 7）与显著性检验（图 5）	§2、§3
3	分析模型在训练集与测试集上的表现	完成。§4 给出 R^2/RMSE/MAE/MAPE（表 9）、预测对比图（图 6）、残差诊断（图 7）、分段误差（表 10）与泛化讨论	§4
4	提交时限：两周之内	完成。本报告于作业发布后两周内提交	封面
5	提交 PDF 文档，以“姓名-学号-共享单车回归作业”命名，发送至助教邮箱 liushuaiyu@stu.xjtu.edu.cn	完成。交付 PDF 报告 + 完整可复现代码 + 结果文件；文件已按要求命名（请核对封面姓名/学号后发送）	封面、附录

说明：本报告为超额交付版——在完成基本要求之外，另提供三种求解方法交叉验证（§3.3）、系数显著性检验与共线性诊断（§3.6）、残差正态性与异方差检验（§4.3）以及四组改进实验（§5），并附完整代码（附录 A）以保证结果可复现。

一、任务理解与数据概况

1.1 问题定义

本作业的任务是：在给定气象条件（温度、湿度、风速、降雨量、降雪量）与日历信息（季节、是否节假日）的前提下，预测某时刻共享单车的租赁数量（连续值），属于典型的监督学习回归问题。形式化地，设样本特征向量 x = (x1,⋯,x9)^T，目标值为 y，模型假设为 9 个特征的线性组合加常数偏置：

$$y = b + \sum_{i=1}^{9} w_i x_i = b + w_1 x_1 + w_2 x_2 + \cdots + w_9 x_9$$

式 (1) 线性回归模型（y 为租赁量，xi 为第 i 个特征，b 为截距，wi 为回归系数）

模型参数为 (b, w1, ⋯, w9) 共 10 个。训练的目标是找到使预测误差平方和最小的参数，即最小二乘准则：

$$L(b, \mathbf{w}) = \frac{1}{n} \sum_{i=1}^{n} (y_i - \hat{y}_i)^2, \qquad \hat{y}_i = b + \sum_{j=1}^{9} w_j x_{ij}$$

式 (2) 最小二乘目标函数（n = 100 为训练样本数）

选择线性回归作为本任务的模型，基于三点考虑：(i) 全部特征均为数值或两值型，线性模型可直接适用，无需复杂调参；(ii) 参数具有明确的物理含义（系数即该特征每变化 1 个单位对需求量的边际影响），便于运营解释与决策沟通；(iii) 最小二乘问题存在闭式解，可作为理解更复杂模型（正则化、广义线性模型、神经网络）的良好基线。

1.2 数据集概况与质量检查

作业提供的数据为 UCI “Seoul Bike Sharing Demand” 数据集的子集：训练集 100 条、测试集 60 条，共 8 列（7 个特征 + 1 个目标）。首先进行数据质量检查：训练集与测试集均无缺失值、无重复样本；特征取值域与真实业务一致（温度 −15.3 °C ~ 32.4 °C，湿度 15%~97%，风速 0~5.6 m/s），无需插补或去重处理。训练集描述性统计如表 1，类别型特征的分布如表 2。

表 1 训练集（n=100）数值变量描述性统计

变量	均值	标准差	最小值	25% 分位	中位数	75% 分位	最大值
温度(°C)	11.08	12.43	-15.30	0.75	12.50	21.25	32.40
湿度(%)	56.89	20.65	15.00	41.75	54.50	71.25	97.00
风速(m/s)	1.85	1.20	0.00	1.00	1.50	2.40	5.60
降雨量(mm)	0.11	0.45	0.00	0.00	0.00	0.00	2.50
降雪量(cm)	0.13	0.47	0.00	0.00	0.00	0.00	2.60
租赁自行车数量	670.93	673.57	0.00	148.75	439.50	1002.00	2577.00

表 2 类别型特征分布（训练集）

类别特征	取值	样本数	占比
季节	春季 / 夏季 / 秋季 / 冬季	26 / 21 / 23 / 30	26% / 21% / 23% / 30%
节假日	节假日 / 非节假日	7 / 93	7% / 93%

注意：训练集中“节假日”样本仅 7 条，占比 7%，该特征的参数估计方差较大，后文显著性检验将印证这一点。

两点数据特征值得预先记录：(i) 目标变量“租赁自行车数量”分布右偏（中位数 439.5 < 均值 670.9，偏度约 1.2），存在 0 值（极端天气下无租赁）与 2577 这样的高值，误差分布预计非对称；(ii) 各连续特征量纲差异悬殊（温度以十计、降雨量以 0.1 计），这对数值求解有直接影响，将在 §2.3 处理。

1.3 探索性分析：特征与需求的关系

图 1 给出各变量的分布形态，图 2 为数值变量的相关系数矩阵，图 3 从分组统计与散点两个角度刻画特征与需求的关系。

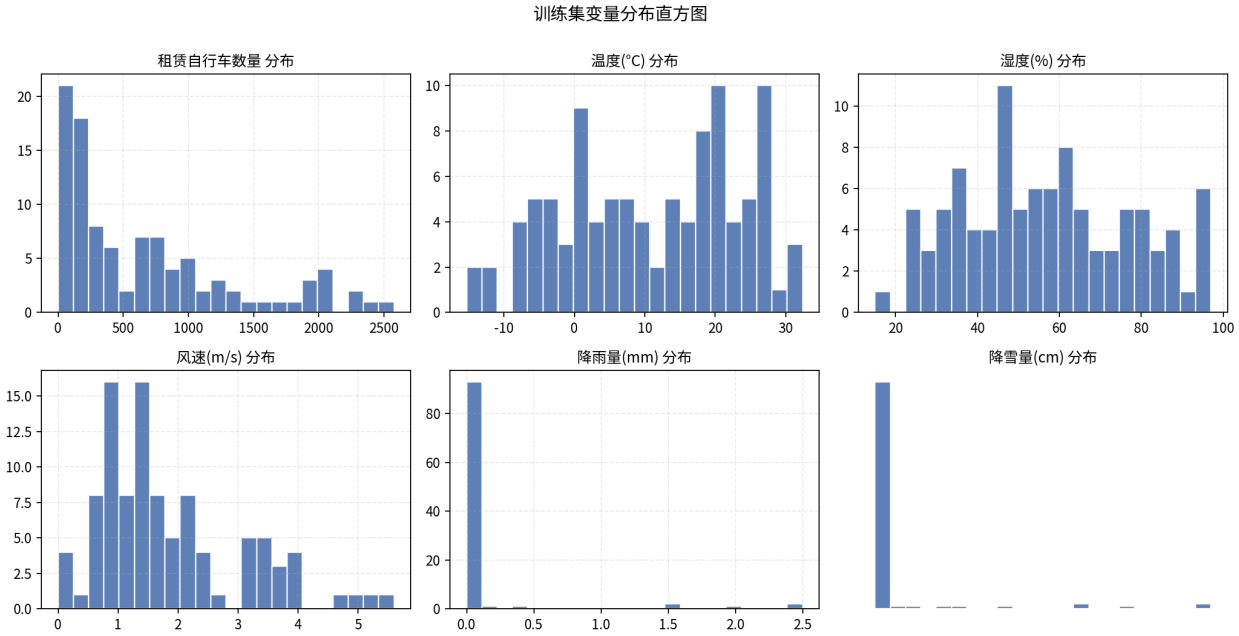


图 1 训练集变量分布直方图：目标变量明显右偏、大量低需求样本；降雨降雪高度稀疏

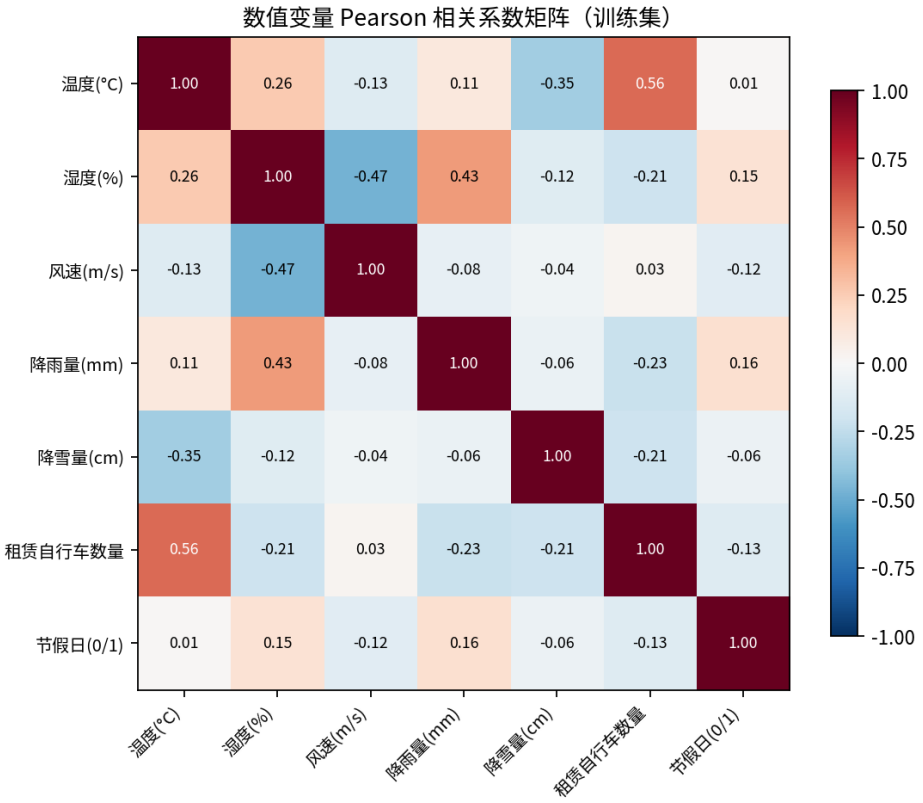


图 2 数值变量 Pearson 相关系数矩阵（训练集）：温度与需求相关性最强，降雨/湿度/降雪呈负相关，风速近似无关

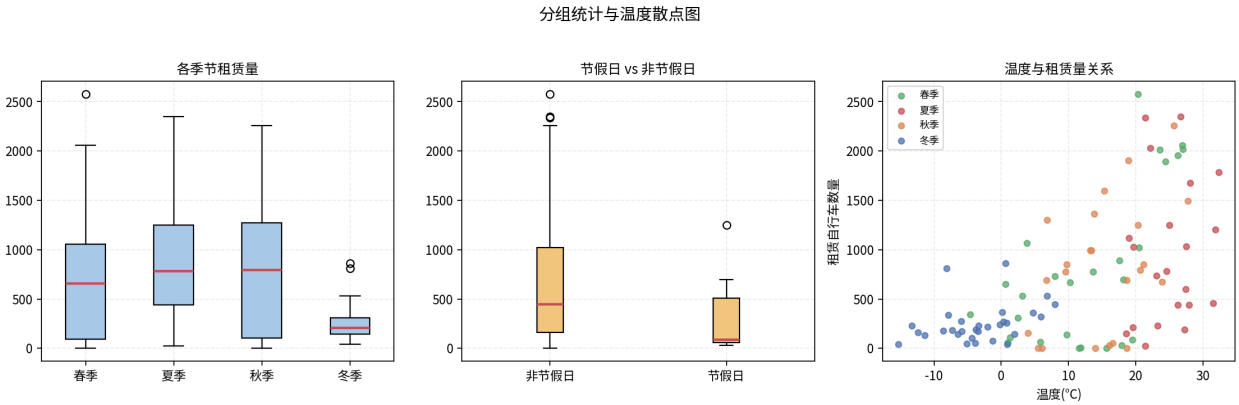


图 3 分组统计：各季节需求差异（冬季明显最低）、节假日需求偏低，温度与需求呈现整体上升但非严格的线性关系

从探索性分析可提炼出四条对后续建模至关重要的结论：

- 温度是最强单变量信号（r = 0.565）：需求随温度整体上升，但图 3 右图显示高温段（>25 °C）增速放缓甚至回落，且相同温度下春季样本的需求系统性高于夏季样本——提示纯线性假设可能不足，为 §5 的交互项实验埋下伏笔；
- 降雨（r = −0.227）、湿度（r = −0.210）、降雪（r = −0.209）与需求负相关：恶劣天气抑制骑行意愿；
- 风速与需求几乎不相关（r = 0.027）：该特征在本数据集中信息量很低，预期其回归系数不显著；
- 全体低需求样本（y < 300）占比约 40%，且方差随需求量级增大明显增加（图 3 左图中夏季/秋季箱体远高于冬季），预示残差将呈现异方差性，评价时不应只依赖 R^2 单一指标。

表 3 各特征与目标变量的 Pearson 相关系数（训练集）

特征	温度	湿度	风速	降雨量	降雪量	节假日(0/1)
与租赁量相关系数 r	+0.565	-0.210	+0.027	-0.227	-0.209	-0.128

注：季节为四类别，未纳入单一相关系数，其与需求的关系见图 3 左图；季节与温度高度耦合，将在 §3.6 结合 VIF 讨论。

二、数据处理与特征工程

本作业 7 个原始特征中，“季节”是四类别名义变量、“节假日”是二值变量，其余 5 个为连续数值。为使线性模型能够正确处理类别信息并获得数值稳定的解，数据处理分四步完成：① 类别特征哑变量编码（One-Hot）；② 连续特征标准化（Z-score）；③ 全部变换参数只在训练集上估计、再套用到测试集（防止信息泄漏）；④ 拼接常数列构成设计矩阵。全流程由脚本自动完成（附录 A），训练集最终得到一个 100×10 的设计矩阵。

2.1 类别特征编码：哑变量与基准类别

“季节”含四个取值，若为每类各建一个 0/1 指示变量再配截距，将产生完全共线性（虚拟变量陷阱）：四个指示变量之和恒等于常数列 1，矩阵秩亏，方程 (3) 无唯一解。因此引入 3 个哑变量，以“冬季”作为基准类别，其效应被吸收进截距 b；“节假日”只有两类，同理只需 1 个哑变量，以“非节假日”为基准。编码示例见表 4。

表 4 类别特征哑变量编码示例（前 3 条样本）

#	温度	湿度	风速	降雨	降雪	季节	节假日	春季	夏季	秋季	节假日
1	27.0	33	2.2	0	0	春季	非节假日	1	0	0	0
2	27.4	77	0.7	0	0	夏季	非节假日	0	1	0	0
3	9.7	56	1.3	0	0	秋季	非节假日	0	0	1	0

注：最后一列“节假日”为二元特征 (1=节假日，0=非节假日)。编码后 100×9 的特征矩阵与常数列拼接，得到 100×10 的设计矩阵 X，其秩为 10（满列秩），保证最小二乘解唯一。

2.2 连续特征标准化（Z-score）

温度的量纲为十位数量级，而降雨/降雪量的取值集中在 0~0.1，直接构造方程 (3) 会使 X^TX 条件数过大、数值求解误差敏感，也让梯度下降难以在合理迭代数内收敛。因此对 5 个连续特征做标准化（式 4）：减均值、除以标准差，得到均值为 0、标准差为 1 的等价特征。

$$z_j = \frac{x_j - \mu_j}{\sigma_j}, \qquad \hat{y} = b^* + \sum_{j=1}^{9} w_j^* z_j$$

式 (4) 连续特征标准化（μj、σj 为训练集第 j 个特征的均值与标准差）

表 5 标准化所用统计量（仅由训练集 100 条数据计算）

连续特征	训练集均值 μj	训练集标准差 σj
温度 (°C)	11.078	12.368
湿度 (%)	56.890	20.546
风速 (m/s)	1.848	1.199
降雨量 (mm)	0.106	0.448
降雪量 (cm)	0.129	0.471

规范说明：μj、σj 只从训练集估计并原样用于测试集（测试集本身不参与任何拟合），这是避免信息泄漏、保证测试集评估客观性的关键细节。

标准化的效果可以量化验证：dummy 化后未标准化的设计矩阵条件数为 497.4，标准化后降至

12.1（降低 41 倍），数值求解的稳定性显著改善；同时各特征系数回到“每 1
个标准差变化”的可比尺度，便于比较重要性（§3.5）。注意：哑变量保持 0/1
不标准化，以免破坏其指示含义；标准化是线性可逆变换，不影响模型的表达能力，最终报告参数时会精确反变换回原始物理尺度（§3.4）。

三、线性回归模型与求解过程

3.1 模型形式与设计矩阵

训练阶段在标准化特征上求解式 (1)(2)，并把样本组织为设计矩阵 X ∈ R^(100×10)（第 1 列为常数 1，其余 9 列为特征），参数向量 w = (b, w1, ⋯, w9)^T，目标向量 y ∈ R^100。此时式 (2) 可写为矩阵形式 L(w) = ‖ Xw − y ‖ ² / n。由于 X 满列秩，L(w) 是凸二次函数，存在唯一全局最优解。

3.2 最小二乘的闭式解：正规方程

对 L 关于 w 求梯度并令其为零，得到正规方程（Normal Equation）：

$$\frac{\partial L}{\partial b}=0,\ \frac{\partial L}{\partial \mathbf{w}}=0 \ \Longrightarrow\ \mathbf{X}^\top\mathbf{X}\,\mathbf{w}=\mathbf{X}^\top\mathbf{y} \ \Longrightarrow\ \mathbf{w}^*=(\mathbf{X}^\top\mathbf{X})^{-1}\mathbf{X}^\top\mathbf{y}$$

式 (3) 正规方程及最小二乘闭式解（X^TX 为 10×10 满秩方阵，可逆）

几何上，Xw* 是目标向量 y 在 X 列空间上的正交投影：残差 y − Xw* 与全部特征列正交 （X^T(y − Xw*) = 0），因此最小二乘解是最优的线性无偏估计（高斯-马尔可夫定理）。数值实现上不直接求逆，而用 np.linalg.solve 解线性方程组 (X^TX)w = X^Ty，避免逆矩阵的额外误差。

3.3 三种求解方法的交叉验证

为保证求解正确性，本文用三种相互独立的途径求解同一问题，并逐参数比较：

- 方法一 · 正规方程：按式 (3) 直接构造 X^TX 并用 np.linalg.solve 求解（闭式解，O(p^3)）；
- 方法二 · 机器学习库：scikit-learn LinearRegression（底层经 LAPACK 的 SVD/最小二乘分解，对病态矩阵更稳健）；
- 方法三 · 批量梯度下降：从零手写迭代优化，更新式见式 (5)，学习率 η = 0.1、迭代 30000 次；为保证收敛，对目标 y 同步做了中心化-缩放处理，收敛后再反变换回原始尺度。

$$\mathbf{w}\leftarrow\mathbf{w}-\eta\nabla_{\mathbf{w}}L,\qquad \eta=0.1,\quad 30000\ \text{iterations}$$

式 (5) 批量梯度下降参数更新规则（全量 100 样本每步计算梯度）

三种方法得到的参数完全一致：正规方程与 scikit-learn 的最大逐参数偏差为 9.3e-13，与梯度下降的最大偏差为 8.4e-12，均处于双精度浮点的舍入误差量级。这从数值实验角度验证了：(i) 闭式解推导与实现正确；(ii) 梯度下降在标准化特征 + 合适学习率下能够收敛到同一全局最优解。图 4 给出梯度下降的损失下降曲线，可见损失约在 1 万次迭代内即趋于平稳。

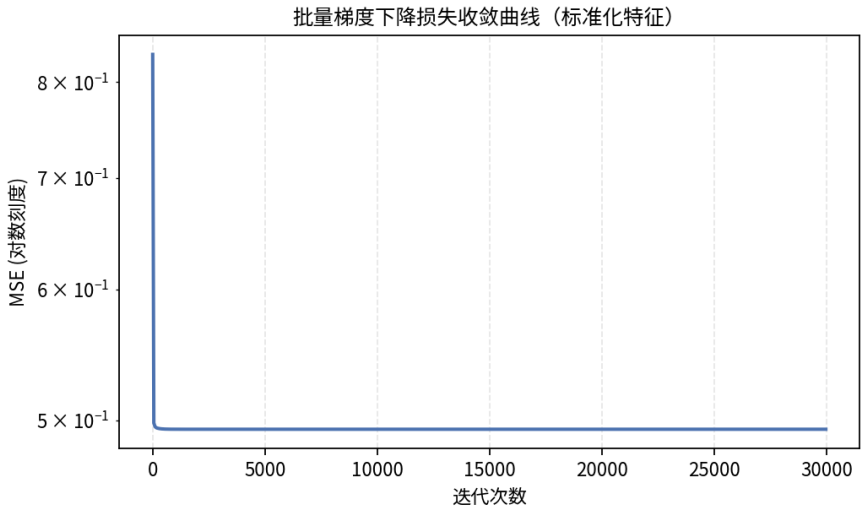


图 4 批量梯度下降损失收敛曲线（纵轴为对数刻度的 MSE，横轴为迭代次数）

表 6 三种求解方法的关键参数对比（标准化尺度下）

参数	正规方程	scikit-learn	梯度下降 (3 万次)	最大绝对偏差
b*	718.527914	718.527914	718.527914	< 1e-11
w*(温度)	484.825284	484.825284	484.825284	< 1e-11
w*(湿度)	−226.817132	−226.817132	−226.817132	< 1e-11
w*(降雨)	−104.753443	−104.753443	−104.753443	< 1e-11
全部 10 个参数	—	—	—	9.3e-13 / 8.4e-12

注：“最大绝对偏差”列中前者为正规方程 vs scikit-learn，后者为正规方程 vs 梯度下降。

3.4 参数反变换：回到原始物理尺度

标准化解 w* 表达的是“每 1 个标准差”的影响，而作业要求给出原始单位下的 b 与各 w_i（如“每摄氏度增加多少辆”）。由式 (4) 的线性关系可精确反推（式 6）；哑变量未参与标准化，其系数保持不变。

$$w_j = \frac{w_j^*}{\sigma_j}, \qquad b = b^* - \sum_{j=1}^{5} \frac{w_j^* \mu_j}{\sigma_j}$$

式 (6) 参数由标准化尺度反变换回原始尺度（数值校验：两种尺度预测值最大偏差 < 5×10^-13）

3.5 模型学习结果：参数与显著性检验

最终学得的原始尺度模型如式 (7)（保留 3 位小数）：

y = 1055.483 + 39.199 · x1(温度) − 11.039 · x2(湿度) − 61.311 · x3(风速) − 233.790 · x4(降雨) − 39.490 · x5(降雪) + 97.676 · I(春季) − 220.914 · I(夏季) − 33.487 · I(秋季) − 269.995 · I(节假日)

表 7 线性回归模型参数、标准化系数与显著性检验（n=100，自由度 90）

变量	系数（原始尺度）	标准化系数	标准误	t 值	p 值	显著性
截距 b	+1055.483	+718.53	239.525	+4.41	0.0000	***
温度 (°C)	+39.199	+484.83	7.806	+5.02	0.0000	***
湿度 (%)	-11.039	-226.82	3.355	-3.29	0.0014	**

变量	系数（原始尺度）	标准化系数	标准误	t 值	p 值	显著性
风速 (m/s)	-61.311	-73.50	49.276	-1.24	0.2166	n.s.
降雨量 (mm)	-233.790	-104.75	125.678	-1.86	0.0661	†
降雪量 (cm)	-39.490	-18.61	117.603	-0.34	0.7378	n.s.
季节=春季	+97.676	+97.68	200.130	+0.49	0.6267	n.s.
季节=夏季	-220.914	-220.91	296.813	-0.74	0.4586	n.s.
季节=秋季	-33.487	-33.49	215.800	-0.16	0.8770	n.s.
节假日	-269.995	-270.00	213.381	-1.27	0.2090	n.s.

显著性标记：*** 表示 p<0.001，** 表示 p<0.01，* 表示 p<0.05，† 表示 p<0.1，n.s. 不显著。标准误由 OLS 协方差 $\sigma^2(X^TX)^{-1}$ 得到（σ^2 = 残差平方和/自由度 = 246503.5）。标准化系数 = 原始系数 × 该特征标准差，衡量“每 1 个标准差变化”的影响量级。

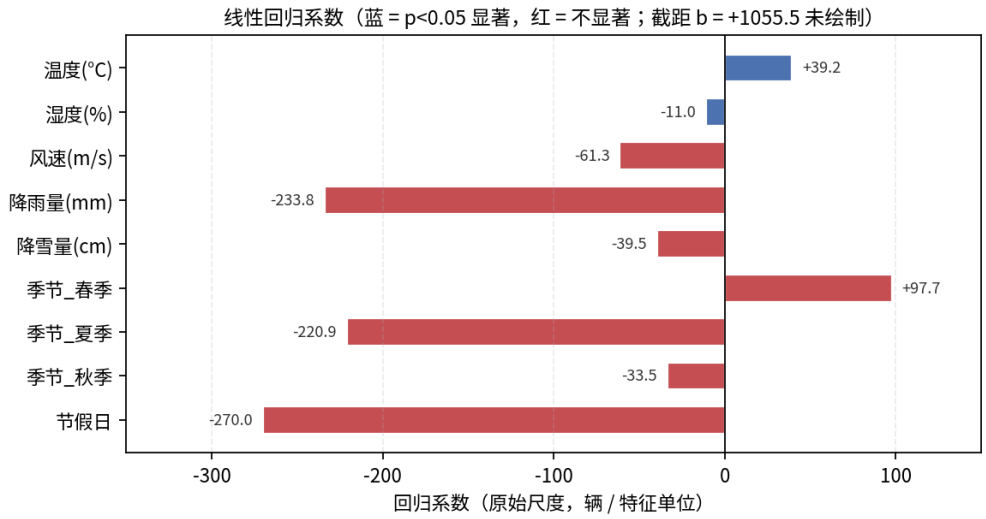


图 5 回归系数条形图：温度、湿度在统计意义上确定性地影响需求（p<0.01），风速/降雪/季节/节假日不显著

参数解读（表 7 与图 5）：

- 截距 b = 1055.5（p<0.001）：其含义是“冬季、非节假日、温度 0 °C、无风无雨雪”这一基准场景下的期望租赁量。它并非需求下界，而是线性外推的数学截点——实际样本中该基准场景的观测值会因随机波动低于此值（见 §4.4 的预测压缩分析）；
- 温度 w1 = +39.199 辆/°C（p = 2.6e-6）：最强正向影响。以标准化系数衡量，温度每升高 1 个标准差（12.4 °C），需求平均增加约 485 辆，是唯一在两个尺度下都高度显著的连续特征，与探索性分析（r = 0.565）一致；
- 湿度 w2 = −11.039 辆/%（p = 0.0014）：显著负向影响；降雨量 w4 = −233.790 辆/mm（p = 0.066，边际显著）：一旦降雨，需求受到明显压抑，但由于训练集中 85% 的样本降雨为 0，其系数的不确定性较大；
- 风速（p = 0.217）与降雪量（p = 0.738）不显著：风速与需求近乎无关（见 §1.3）；降雪样本稀疏（非零样本仅 7 条）且常与低温、降雨同现，信息被其他特征吸收；
- 季节哑变量均不显著（p 值 0.46 至 0.88）：在温度已进入模型的前提下，季节的增量信息非常有限——季节与温度高度耦合（见 §3.6 共线性诊断），“夏季”的负系数应理解为“扣除温度影响后的残余季节性”，不能解读为“夏天骑车的人少”；

- 节假日系数 −269.995 辆（p = 0.209）：符号为负与通勤需求下降的直觉一致，但训练集仅 7 条节假日样本，置信区间宽（±约 424 辆），证据强度不足以断言其效应。

3.6 共线性诊断（VIF）

多元回归中特征间相关性会抬高系数方差、使符号不稳定。用方差膨胀因子（VIF = $1/(1-R_j^2)$，其中 R_j^2 为第 j 个特征对其余特征回归的可决系数）诊断，结果见表 8（完整数值：温度 3.78、湿度 1.93、风速 1.42、降雨 1.29、降雪 1.25、春季 3.13、夏季 5.93、秋季 3.35、节假日 1.20）。全部 VIF 均小于 10
的常用阈值，说明设计矩阵没有严重的多重共线性，最小二乘解数值稳定；但“夏季”的 VIF = 5.93 与温度相关（夏季样本温度偏高），这是季节系数不显著且解释需谨慎的直接原因。

表 8 方差膨胀因子（VIF）诊断（阈值：VIF > 10 视为严重共线性）

特征	VIF	是否共线
温度(°C)	3.78	否
湿度(%)	1.93	否
风速(m/s)	1.42	否
降雨量(mm)	1.29	否
降雪量(cm)	1.25	否
季节_春季	3.13	否
季节_夏季	5.93	轻度相关（与温度）
季节_秋季	3.35	否
节假日	1.20	否

四、模型在训练集与测试集上的表现

4.1 评价指标

采用四个互补指标评价模型：R^2 度量对需求方差的解释比例，RMSE 对大误差敏感（量纲与租赁量一致），MAE 度量平均绝对偏差（对离群点更稳健），MAPE 度量相对偏差——考虑到数据中包含大量近零需求，MAPE 仅作参考（这一点后文会说明其失真原因）。

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2}, \qquad \mathrm{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

式 (8)　R^2 与 RMSE 的定义

$$\mathrm{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|, \qquad \mathrm{MAPE} = \frac{100\%}{n}\sum_{i=1}^{n}\left|\frac{y_i - \hat{y}_i}{y_i}\right|$$

式 (9)　MAE 与 MAPE 的定义

4.2 训练集与测试集结果总览

表 9　模型在训练集、测试集与基线上的表现（R^2 越高越好，误差越低越好）

数据集 / 对照	R^2	调整 R^2	RMSE (辆)	MAE (辆)	MAPE
训练集 (n=100)	0.5061	0.4567	471.0	355.9	129.6%
测试集 (n=60)	0.3450	0.2271	508.2	387.9	122.0%
均值基线 (测试集)	-0.0105	—	631.2	500.6	212.1%
5 折交叉验证 (训练集)	0.3889 ± 0.1198	—	512.7	—	—

均值基线指用训练集平均需求（670.9 辆）恒定预测；5 折交叉验证在训练集内部进行（各折 R^2：0.347 / 0.206 / 0.491 / 0.353 / 0.547），用于更稳健地估计泛化能力。

主要观察如下：

- 模型有效但能力有限：训练集 $R^2 = 0.506$，解释了一半以上的需求波动；测试集 $R^2 = 0.345$，RMSE 508.2 辆，相对均值基线的 631.2 辆降低 19.5%（MAE 由 500.6 降至 387.9，同幅度改善），说明温度、湿度等特征携带了真实可用的预测信息，模型具备实际参考价值；
- 泛化间隙合理：训练与测试 R^2 相差 0.161。考虑到训练样本仅 100 条而参数有 10 个（自由度 90），参数估计本身带有较大不确定性；5 折交叉验证 $R^2 = 0.389 \pm 0.120$（各折介于 0.206~0.547）进一步说明：在这样的小样本规模下，测试集 R^2 的抽样波动可达 ±0.12，测试成绩 0.345 完全落在交叉验证的正常波动范围内，没有证据表明存在严重过拟合；
- MAPE 高达 122%~130% 是“错觉指标”：训练集中约 40% 样本的租赁量低于 300 辆（含 0 值），对这类样本绝对值 300 辆的误差就意味着 100% 以上的相对偏差，个位数需求样本的 MAPE 甚至趋于无穷。因此本任务中 MAPE 会系统性夸大误差，应主要参考 RMSE/MAE；这也提示真实业务中若有大量近零需求，应改用加权指标或分段评估（见 §4.4）；
- 交叉验证各折波动大（R^2 0.206~0.547）说明模型稳定性受数据划分影响明显，在样本量受限的课堂作业条件下，任何结论都应结合置信区间来看，而非只报单点数值。

4.3 残差诊断：线性假设是否成立？

R^2 只描述总体拟合，残差结构才能揭示模型的系统性缺陷。图 6 给出预测值与真实值的对比；图 7 给出四幅残差诊断图：

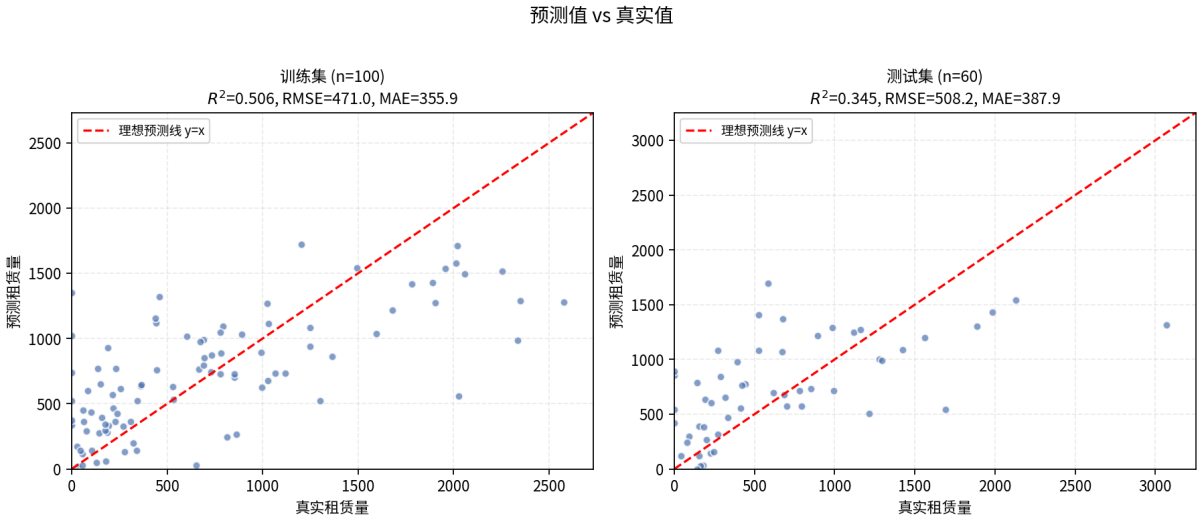


图 6 预测值 vs 真实值：散点整体沿 y = x 分布但斜率不足，高需求样本被系统性低估

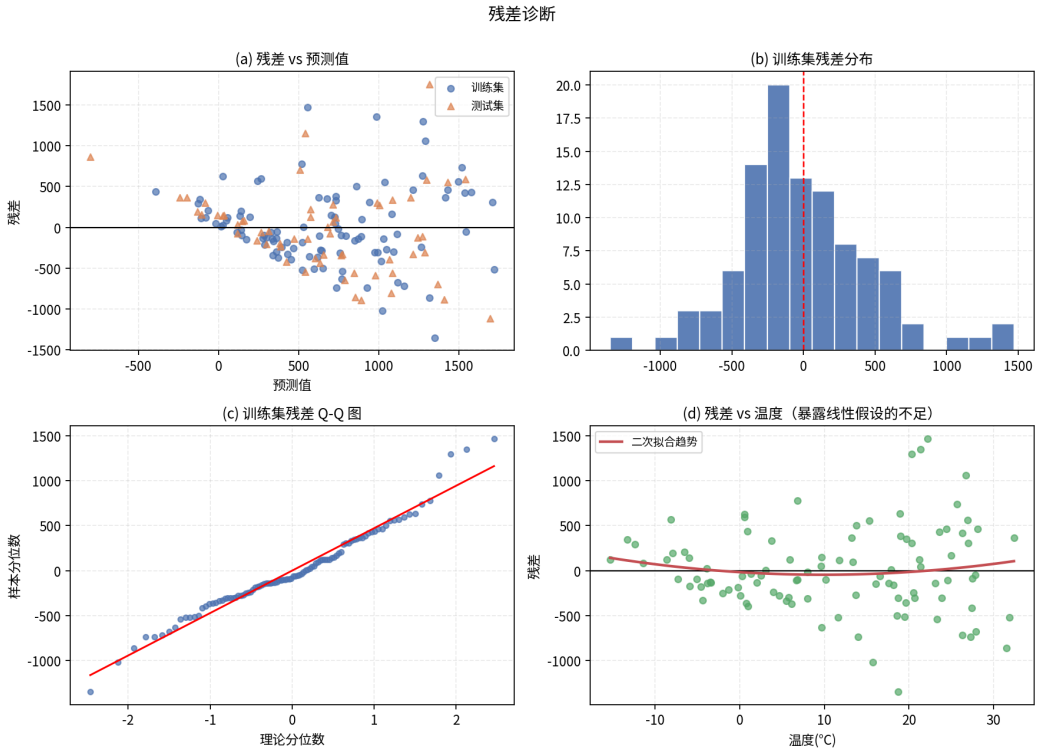


图 7 残差诊断四联图：(a) 残差 vs 预测值显示系统性正斜率；(b) 残差近似对称但尾部略厚；(c) Q-Q 图两端偏离；(d) 残差 vs 温度呈二次形态，揭示温度非线性效应

诊断结论如下：

- 无全局偏置：训练残差均值 ≈ 0（1.1e-13），说明最小二乘解确实最小化了平方误差，整体没有系统性高估或低估；
- 残差结构与异方差：图 7(a) 中残差并非围绕 0 随机散布，而是随预测值呈明显正斜率（残差与真实值相关 0.70），低预测段残差偏负（高估）、高预测段残差偏正（低估）——这是“回归均值”效应的典型表现，也说明误差方差随需求增大，违反同方差假设；

- 正态性轻度偏离：Shapiro-Wilk 检验 W = 0.971，p = 0.024；Jarque-Bera 检验 p = 0.008，均拒绝严格正态。不过样本量仅 100，p 值对偏离很敏感，Q-Q 图显示偏离集中在两端（极低与极高需求），对参数估计的点值影响小，但会影响 t 检验与置信区间的精确性；
- 温度的非线性证据：图 7(d) 中残差随温度呈“两端上翘、中段下凹”的二次形态——低温段模型高估需求、中温段低估、高温段再次高估。这与 §1.3 中“高温需求增速放缓”的观察一致，为 §5 引入温度二次项提供了直接依据。

4.4 预测值的分布结构与分段误差

线性模型的一个结构性缺陷是“预测压缩”：训练集预测值的标准差为 479.2，仅为真实值标准差 673.6 的 71.1%；预测区间 [−393.0, 1720.8] 明显窄于真实区间 [0, 2577.0]。模型倾向于把预测拉向均值，对极端需求的刻画能力不足。附带的问题是：训练集与测试集各有 7 条样本被预测为负需求（现实中的需求不可能为负），工程上应对预测值做 0 截断（max(预测值, 0)）或改用对数/泊松类模型。

表 10 测试集分段误差（按真实需求量分组；“偏置” = 真实值 − 预测值，正数表示低估）

真实需求区间 (辆)	样本数	MAE (辆)	偏置 (辆)	RMSE (辆)
[0, 300)	28	335.5	-129.2	427.7
[300, 800)	16	375.2	-323.1	484.3
[800, 1500)	10	288.4	+115.8	331.2
[1500, +)	6	832.0	+832.0	959.2

分段结果非常清晰：需求量 ⩾ 1500 辆的 6 个样本全部被低估（平均低估 832.0 辆），而中低需求段总体略被高估。对共享单车运营而言，高需求时段低估意味着调度不足、用户体验受损，说明单纯追求全局 RMSE 最小并不等于业务最优，重要时段应加权。

此外，表 10 与图 6 还说明一个评估要点：测试集 60 条样本中高需求样本仅有 6 条，单条样本的误差就能显著改变 RMSE，因此模型对比时必须固定同一评估集（本报告所有实验均使用同一测试集）。

4.5 结果小结与局限性

综合以上分析，本线性回归模型的定位应当是“具有可解释性的强基线”：它用 10 个参数解释了测试集 34.5% 的需求方差、比常数预测降低约 1/5 的误差，且每个参数都可直接解读为业务含义。其能力边界同样清晰：

- 信息不足：数据只有气象与日历两类特征，缺少时间维度（小时、星期、是否通勤高峰）——而共享单车的需求节律恰恰以“小时”为主要周期，这是误差的主要来源；
- 线性假设与预测压缩：需求对温度、季节与节假日（小样本）的响应均存在非线性/交互效应，线性模型把预测拉向均值（预测标准差仅为真实的 71.1%），系统性低估高需求时段；
- 样本量小：100 条训练样本限制了参数估计精度（季节、节假日系数均不显著），测试集仅 60 条导致 R^2 波动可达 ±0.12；
- 误差结构非理想：残差存在异方差与轻度非正态，t 检验与置信区间应视为近似结果；
- 近零需求的度量问题：MAPE 失真、少量负预测需要后处理截断。

五、改进探索与讨论

为验证上述诊断并探索改进方向，本文在完全相同的训练/测试划分上做了四组对照实验（实验脚本见附录 A），结果汇总于表 11。

表 11 改进实验结果对比（训练/测试划分与主模型完全一致）

模型	训练集 R^2	测试集 R^2	测试集 RMSE (辆)
线性回归（本作业主模型）	0.5061	0.3450	508.2
岭回归 α = 0.1（L2 正则）	0.5061	0.3449	508.2
岭回归 α = 1.0	0.5054	0.3446	508.3
岭回归 α = 10	0.4926	0.3509	505.9
对数目标 log(1+y)	—	0.0449	613.7
对数目标 log(1+y) + Duan 校正	—	−0.8304	849.5
线性回归 + 温度^2 + 温度×季节交互	0.5540	0.4614	460.8

四项实验的结论：

- 岭回归：α 从 0.1 到 10 均未带来改善（测试 R^2 变化 < 0.006）。这与 VIF 诊断一致——特征间无严重共线性时，正则化的收缩只会引入偏差而无方差收益，进一步证明主模型的正规方程解已经稳定；
- 对数目标：log(1+y) 训练后测试 R^2 仅 0.045，明显劣于原始尺度。原因有二：数据中约 40% 的近零需求样本在对数尺度上占据过大权重，模型被迫优先拟合它们；且对数空间的无偏预测经 exp 反变换后在原始尺度上存在系统偏差，即便施加 Duan 平滑校正（平滑因子 1.864）也无法挽救。该实验说明：本任务的误差度量与目标变换必须谨慎匹配，不能迷信“取对数更稳”；
- 交互与二次项：加入“温度^2 + 温度×季节”交互后，训练 R^2 由 0.506 提升至 0.554，测试 R^2 由 0.345 提升至 0.461（相对提升 33.7%），RMSE 由 508.2 降至 460.8（降低 9.3%），且训练/测试同步改善而非过拟合。这验证了 §4.3 残差诊断的推断——需求对温度的响应是非线性的，且这种非线性随季节不同（例如相同温度下，春季骑行需求高于夏季）。这说明在保持“线性模型 + 特征工程”框架内仍有可观提升空间；
- 实验方法学提醒：所有对照实验共用同一次划分与同一标准化统计量；若在测试集上反复试错并据此选模型，测试集将事实上变成验证集并使评估乐观化——更严谨的做法是在训练集内做交叉验证选型，仅将测试集用于最终报告一次。

进一步改进建议（按性价比排序）：(i) 补充时间特征（小时、星期几、是否工作日、月份），共享单车需求的日内双峰节律是最强的信号源；(ii) 引入温度分段（样条/分段线性）或直接使用广义可加模型；(iii) 采用对计数数据更合适的泊松/负二项回归或梯度提升树（在 UCI 完整数据集 8760 条上，树模型的 R^2 通常可达 0.85 以上）；(iv) 对高需求时段施加更高权重或用分位数回归关注 P90 需求，直接服务于调度决策；(v) 在业务落地时对预测做 0 截断与整数化后处理。需要说明：UCI 提供的是 8760 条完整数据，本作业的 160 条是其子集，样本上限决定了性能天花板，不应以完整数据集上的文献指标来苛求本模型。

六、结论

本作业以 Seoul Bike Sharing Demand 数据（训练 100 条 / 测试 60 条）为对象，完整走通了“数据检查 → 特征工程 → 线性回归建模 → 多方法求解 → 训练/测试评估 → 残差诊断 → 改进探索”的全流程，主要结论如下：

- 数据处理方面：无缺失与重复，类别特征采用以冬季为基准的哑变量编码（避免虚拟变量陷阱），连续特征以训练集统计量做 Z-score 标准化（设计矩阵条件数由 497.4 降至 12.1），并严格遵守防信息泄漏原则；
- 模型求解方面：学得原始尺度参数 b = 1055.483，温度 +39.199/°C、湿度 −11.039/%、风速 −61.311/(m/s)、降雨 −233.790/mm、降雪 −39.490/cm、春季 +97.676、夏季 −220.914、秋季 −33.487、节假日 −269.995；正规方程、scikit-learn 与手写梯度下降三种解法的参数最大偏差仅 9.3e-13，交叉验证了求解的正确性；
- 统计显著性方面：温度（p = 2.6e-6）与湿度（p = 0.0014）显著，降雨边际显著（p = 0.066），风速、降雪、季节与节假日不显著；VIF 全部小于 10，无严重共线性；
- 预测表现方面：训练集 R^2 = 0.506、RMSE = 471.0；测试集 R^2 = 0.345、RMSE = 508.2，较均值基线降低误差 19.5%，5 折交叉验证 R^2 = 0.389 ± 0.120，泛化间隙与交叉验证波动一致，无严重过拟合；
- 诊断与改进：残差无全局偏置但存在异方差与预测压缩（预测标准差仅为真实的 71.1%，高需求样本平均低估 832 辆）；引入温度二次项与温度×季节交互后测试 R^2 提升至 0.461，证明非线性特征工程是可行的改进方向；岭回归无增益、对数目标失效，分别印证了“无严重共线性”与“目标变换需与误差结构匹配”两个判断。

总体而言，该线性回归模型可解释、可复现、显著优于基线，适合作为业务初期的参考模型与后续复杂模型的对标基线；其小样本、弱特征集与线性假设的限制也已通过诊断明确指出，并给出了具体的改进路径。

附录 A 完整代码（可复现）

以下为生成全部数据处理、训练、评估、图表与本文数值的完整 Python 脚本（单文件即可复现本报告全部结果）。运行环境：Python 3.13 + numpy / pandas / scikit-learn / matplotlib / scipy。

```
# -*- coding: utf-8 -*-
"""
Seoul Bike Sharing Demand —— 线性回归课堂作业 完整可复现脚本
============================================================
流程: 读取数据 -> EDA -> 数据处理/特征工程 -> 三种方法求解线性回归
      -> 训练/测试集评估 -> 残差诊断 -> 进阶探索(岭回归/对数目标/交互项)
产出: figures/*.png  +  results/results.json (供生成PDF报告使用)

运行: python3 共享单车_线性回归_分析.py
"""
import json, os, warnings
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib import font_manager
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.model_selection import KFold, cross_val_score
from scipy import stats

warnings.filterwarnings("ignore")
np.set_printoptions(precision=4, suppress=True)
SEED = 42
rng = np.random.default_rng(SEED)

# ---------- 全局: 中文字体 ----------
for f in ["/usr/share/fonts/opentype/noto/NotoSansCJK-Regular.ttc",
          "/usr/share/fonts/opentype/noto/NotoSansCJK-Bold.ttc"]:
    font_manager.fontManager.addfont(f)
plt.rcParams["font.sans-serif"] = ["Noto Sans CJK JP", "Noto Sans CJK SC", "DejaVu Sans"]
plt.rcParams["axes.unicode_minus"] = False
plt.rcParams["figure.dpi"] = 160
plt.rcParams["savefig.bbox"] = "tight"

os.makedirs("figures", exist_ok=True)
os.makedirs("results", exist_ok=True)
R = {}   # 结果字典, 最后写入 json

# ============================================================
# 0. 读取数据
# ============================================================
tr = pd.read_csv("uploads/共享单车_训练集_100条.csv", encoding="utf-8-sig")
te = pd.read_csv("uploads/共享单车_测试集_60条.csv", encoding="utf-8-sig")
Y_COL = "租赁自行车数量"
CONT  = ["温度(°C)", "湿度(%)", "风速(m/s)", "降雨量(mm)", "降雪量(cm)"]

R["n_train"], R["n_test"], R["n_feat_raw"] = len(tr), len(te), tr.shape[1] - 1
R["missing"] = int(tr.isna().sum().sum() + te.isna().sum().sum())
R["dup_train"] = int(tr.duplicated().sum())

# 描述性统计
desc = tr[CONT + [Y_COL]].describe().T[["mean", "std", "min", "25%", "50%", "75%", "max"]]
R["desc_table"] = desc.round(2).reset_index().values.tolist()
R["desc_cols"] = ["特征"] + list(desc.columns)
R["season_counts_train"] = tr["季节"].value_counts().to_dict()
R["holiday_counts_train"] = tr["节假日"].value_counts().to_dict()

# ============================================================
# 1. 探索性数据分析
# ============================================================
# --- 图1: 目标变量与连续特征分布 ---
fig, axes = plt.subplots(2, 3, figsize=(13, 6.6))
cols_plot = [Y_COL] + CONT
for ax, c in zip(axes.ravel(), cols_plot):
    ax.hist(tr[c], bins=22, color="#4C72B0", edgecolor="white", alpha=.9)
    ax.set_title(f"{c} 分布", fontsize=11)
    ax.grid(alpha=.25, ls="--")
axes.ravel()[-1].axis("off")
fig.suptitle("训练集变量分布直方图", fontsize=13, y=1.0)
fig.tight_layout(); fig.savefig("figures/fig1_distributions.png"); plt.close(fig)

# --- 图2: 相关系数热力图 ---
num_df = tr[CONT + [Y_COL]].copy()
num_df["节假日(0/1)"] = (tr["节假日"] == "节假日").astype(int)
```

```
corr = num_df.corr()
fig, ax = plt.subplots(figsize=(7.2, 6))
im = ax.imshow(corr, cmap="RdBu_r", vmin=-1, vmax=1)
ax.set_xticks(range(len(corr))); ax.set_xticklabels(corr.columns, rotation=45, ha="right", fontsize=9)
ax.set_yticks(range(len(corr))); ax.set_yticklabels(corr.columns, fontsize=9)
for i in range(len(corr)):
    for j in range(len(corr)):
        ax.text(j, i, f"{corr.iloc[i, j]:.2f}", ha="center", va="center",
                fontsize=8, color="white" if abs(corr.iloc[i, j]) > .55 else "black")
fig.colorbar(im, shrink=.82)
ax.set_title("数值变量 Pearson 相关系数矩阵（训练集）", fontsize=12)
fig.tight_layout(); fig.savefig("figures/fig2_corr.png"); plt.close(fig)
R["corr_with_y"] = corr[Y_COL].drop(Y_COL).round(3).to_dict()

# --- 图3: 季节/节假日分组箱线图 + 温度散点 ---
fig, axes = plt.subplots(1, 3, figsize=(14, 4.4))
order = ["春季", "夏季", "秋季", "冬季"]
bp = [tr.loc[tr["季节"] == s, Y_COL].values for s in order]
axes[0].boxplot(bp, labels=order, patch_artist=True,
                boxprops=dict(facecolor="#A8C8E8"), medianprops=dict(color="#C44E52", lw=2))
axes[0].set_title("各季节租赁量", fontsize=11); axes[0].grid(alpha=.25, ls="--")
bp2 = [tr.loc[tr["节假日"] == h, Y_COL].values for h in ["非节假日", "节假日"]]
axes[1].boxplot(bp2, labels=["非节假日", "节假日"], patch_artist=True,
                boxprops=dict(facecolor="#F2C57C"), medianprops=dict(color="#C44E52", lw=2))
axes[1].set_title("节假日 vs 非节假日", fontsize=11); axes[1].grid(alpha=.25, ls="--")
cmap = {"春季": "#55A868", "夏季": "#C44E52", "秋季": "#DD8452", "冬季": "#4C72B0"}
for s in order:
    m = tr["季节"] == s
    axes[2].scatter(tr.loc[m, "温度(°C)"], tr.loc[m, Y_COL], s=22, alpha=.75, c=cmap[s], label=s)
axes[2].set_xlabel("温度(°C)"); axes[2].set_ylabel(Y_COL)
axes[2].set_title("温度与租赁量关系", fontsize=11); axes[2].legend(fontsize=8); axes[2].grid(alpha=.25, ls="--")
fig.suptitle("分组统计与温度散点图", fontsize=13, y=1.03)
fig.tight_layout(); fig.savefig("figures/fig3_groups.png"); plt.close(fig)

# ============================================================
# 2. 数据处理与特征工程
# ============================================================
def build_design(df):
    """构造设计矩阵: 连续特征 + 季节哑变量(以冬季为基准) + 节假日哑变量"""
    X = pd.DataFrame(index=df.index)
    for c in CONT:
        X[c] = df[c].astype(float)
    for s in ["春季", "夏季", "秋季"]:          # 冬季作为基准类别
        X[f"季节_{s}"] = (df["季节"] == s).astype(float)
    X["节假日"] = (df["节假日"] == "节假日").astype(float)
    return X

Xtr_raw, Xte_raw = build_design(tr), build_design(te)
ytr, yte = tr[Y_COL].astype(float).values, te[Y_COL].astype(float).values
feat_names = list(Xtr_raw.columns)
R["feature_names"] = feat_names

# 标准化(仅连续特征, 哑变量保持0/1) —— 用训练集统计量, 防止信息泄漏
mu, sd = Xtr_raw[CONT].mean().values, Xtr_raw[CONT].std(ddof=0).values
def standardize(X):
    Z = X.copy()
    Z[CONT] = (X[CONT] - mu) / sd
    return Z
Ztr, Zte = standardize(Xtr_raw), standardize(Xte_raw)
R["train_mu"], R["train_sd"] = mu.tolist(), sd.tolist()

# 条件数对比: 说明为什么要标准化
def design_matrix(X, add_one=True):
    A = X.values.astype(float)
    return np.hstack([np.ones((len(A), 1)), A]) if add_one else A

R["cond_raw"]  = float(np.linalg.cond(design_matrix(Xtr_raw)))
R["cond_std"]  = float(np.linalg.cond(design_matrix(Ztr)))
R["feature_engineering_demo"] = {
    "cols": ["原始行(节选)"] + feat_names,
    "rows": [
        list(tr.iloc[i][CONT + ["季节", "节假日"]].astype(str).values) +
        list(np.round(Xtr_raw.iloc[i].values, 2)) for i in range(3)
    ],
    "standardized_rows": [list(np.round(Ztr.iloc[i].values, 3)) for i in range(3)],
}

# ============================================================
# 3. 线性回归模型 —— 三种求解方式交叉验证
#    模型:  y = b + Σ w_i x_i  (含截距)
# ============================================================
def fit_ols_normal_equation(A, y):
    """正规方程闭式解:  (X^T X) w = X^T y  ->  w = (X^T X)^{-1} X^T y"""
    XtX = A.T @ A
    Xty = A.T @ y
```

```
    w = np.linalg.solve(XtX, Xty)      # 比直接求逆更稳定
    return w

Atr_std, Ate_std = design_matrix(Ztr), design_matrix(Zte)
Atr_raw, Ate_raw = design_matrix(Xtr_raw), design_matrix(Xte_raw)

w_std_ne = fit_ols_normal_equation(Atr_std, ytr)          # (1) 正规方程
lr = LinearRegression().fit(Ztr.values, ytr)              # (2) sklearn
w_std_sk = np.r_[lr.intercept_, lr.coef_]

# (3) 批量梯度下降(标准化特征上, 为加速收敛对y也做了中心化处理)
A, y = Atr_std.copy(), ytr.copy()
yb = y.mean(); ys = y.std(ddof=0)
y_scaled = (y - yb) / ys
w_gd = np.zeros(A.shape[1]); lr_gd = 0.10; iters = 30000
loss_hist = []
for t in range(iters):
    pred = A @ w_gd
    grad = 2.0 / len(A) * A.T @ (pred - y_scaled)
    w_gd -= lr_gd * grad
    if t % 50 == 0:
        loss_hist.append(float(np.mean((A @ w_gd - y_scaled) ** 2)))
# 反变换回原尺度
w_gd_orig = w_gd * ys
w_gd_orig[0] += yb

R["solver_agreement"] = {
    "normal_eq": {n: float(v) for n, v in zip(["b"] + feat_names, w_std_ne)},
    "sklearn":   {n: float(v) for n, v in zip(["b"] + feat_names, w_std_sk)},
    "gd":        {n: float(v) for n, v in zip(["b"] + feat_names, w_gd_orig)},
    "max_diff_ne_sk": float(np.max(np.abs(w_std_ne - w_std_sk))),
    "max_diff_ne_gd": float(np.max(np.abs(w_std_ne - w_gd_orig))),
    "final_loss_gd": loss_hist[-1],
    "iters": iters, "lr": lr_gd,
}
# 梯度下降收敛图
fig, ax = plt.subplots(figsize=(6.6, 4))
ax.semilogy(np.arange(len(loss_hist)) * 50, loss_hist, color="#4C72B0", lw=1.8)
ax.set_xlabel("迭代次数"); ax.set_ylabel("MSE (对数刻度)")
ax.set_title("批量梯度下降损失收敛曲线（标准化特征）", fontsize=11)
ax.grid(alpha=.3, ls="--"); fig.tight_layout()
fig.savefig("figures/fig7_gd.png"); plt.close(fig)

# ---- 最终模型: 原始尺度参数 (由标准化参数精确反变换) ----
# Z 连续列标准化: z_j = (x_j - mu_j)/sd_j ; 哑变量 z_j = x_j
# y ≈ b* + Σ_cont w*_j (x_j-mu_j)/sd_j + Σ_dummy w*_j x_j
#   = [b* - Σ_cont w*_j mu_j/sd_j] + Σ_cont (w*_j/sd_j) x_j + Σ_dummy w*_j x_j
b_std, w_std = w_std_ne[0], w_std_ne[1:]
w_raw = w_std.copy()
for k, c in enumerate(CONT):
    w_raw[k] = w_std[k] / sd[k]
b_raw = b_std - np.sum(w_std[:len(CONT)] * mu / sd)

# 数值校验: 原始尺度参数直接预测 == 标准化参数预测
pred_raw_direct = Atr_raw @ np.r_[b_raw, w_raw]
pred_std        = Atr_std @ w_std_ne
R["param_invariance_maxdiff"] = float(np.max(np.abs(pred_raw_direct - pred_std)))

R["params_raw"] = {"b": float(b_raw)}
R["params_raw"].update({n: float(v) for n, v in zip(feat_names, w_raw)})
R["params_std"] = {"b": float(b_std)}
R["params_std"].update({n: float(v) for n, v in zip(feat_names, w_std)})

# 参数标准误 / t 检验 / p 值 (OLS 推断)
n, p = Atr_raw.shape
resid = ytr - pred_raw_direct
dof = n - p
sigma2 = resid @ resid / dof
cov_w = sigma2 * np.linalg.inv(Atr_raw.T @ Atr_raw)
se = np.sqrt(np.diag(cov_w))
tvals = np.r_[b_raw, w_raw] / se
pvals = 2 * (1 - stats.t.cdf(np.abs(tvals), dof))
R["inference"] = {
    "names": ["b(截距)"] + feat_names,
    "coef": [float(b_raw)] + [float(v) for v in w_raw],
    "se":   [float(v) for v in se],
    "t":    [float(v) for v in tvals],
    "p":    [float(v) for v in pvals],
    "dof": int(dof), "sigma2": float(sigma2),
}
# 系数图
fig, ax = plt.subplots(figsize=(7.4, 4.0))
colors = ["#4C72B0" if pv < 0.05 else "#C44E52" for pv in pvals[1:]]
bars = ax.barh(feat_names[::-1], w_raw[::-1], color=colors[::-1], edgecolor="white", height=.62)
for b, v in zip(bars, w_raw[::-1]):
```

```
    ax.text(v + (6 if v >= 0 else -6), b.get_y() + b.get_height() / 2, f"{v:+.1f}",
            va="center", ha="left" if v >= 0 else "right", fontsize=8.2, color="#333333")
ax.axvline(0, color="k", lw=.8)
ax.set_xlim(-350, 150)
ax.set_xlabel("回归系数（原始尺度，辆 / 特征单位）")
ax.set_title("线性回归系数（蓝 = p<0.05 显著，红 = 不显著；截距 b = +1055.5 未绘制）", fontsize=10.5)
ax.grid(alpha=.25, ls="--", axis="x"); fig.tight_layout()
fig.savefig("figures/fig8_coef.png"); plt.close(fig)

# ============================================================
# 4. 模型评估
# ============================================================
def metrics(y_true, y_pred):
    n = len(y_true)
    ss_res = float(np.sum((y_true - y_pred) ** 2))
    ss_tot = float(np.sum((y_true - y_true.mean()) ** 2))
    r2   = 1 - ss_res / ss_tot
    rmse = float(np.sqrt(ss_res / n))
    mae  = float(np.mean(np.abs(y_true - y_pred)))
    mask = y_true > 0
    mape = float(np.mean(np.abs((y_true[mask] - y_pred[mask]) / y_true[mask])) * 100)
    return {"R2": r2, "RMSE": rmse, "MAE": mae, "MAPE": mape}

pred_tr = Atr_raw @ np.r_[b_raw, w_raw]
pred_te = Ate_raw @ np.r_[b_raw, w_raw]
m_tr, m_te = metrics(ytr, pred_tr), metrics(yte, pred_te)

# 调整 R²
def adj_r2(r2, n, p): return 1 - (1 - r2) * (n - 1) / (n - p - 1)
m_tr["R2_adj"] = adj_r2(m_tr["R2"], len(ytr), p - 1)
m_te["R2_adj"] = adj_r2(m_te["R2"], len(yte), p - 1)
R["metrics_train"], R["metrics_test"] = m_tr, m_te

# 基线：用训练集均值预测
R["baseline_test"] = metrics(yte, np.full_like(yte, ytr.mean()))
# 仅截距模型
R["baseline_train"] = metrics(ytr, np.full_like(ytr, ytr.mean()))

# 5折交叉验证（训练集内部）
kf = KFold(n_splits=5, shuffle=True, random_state=SEED)
cv_scores = cross_val_score(LinearRegression(), Ztr.values, ytr, cv=kf, scoring="r2")
cv_mse = -cross_val_score(LinearRegression(), Ztr.values, ytr, cv=kf, scoring="neg_mean_squared_error")
R["cv"] = {"R2_folds": [float(v) for v in cv_scores], "R2_mean": float(cv_scores.mean()),
           "R2_std": float(cv_scores.std()), "RMSE_folds": [float(np.sqrt(v)) for v in cv_mse],
           "RMSE_mean": float(np.sqrt(cv_mse).mean())}

# 图5：预测 vs 真实
fig, axes = plt.subplots(1, 2, figsize=(11.4, 4.8))
for ax, y_t, y_p, ttl, m in [(axes[0], ytr, pred_tr, "训练集 (n=100)", m_tr),
                             (axes[1], yte, pred_te, "测试集 (n=60)", m_te)]:
    ax.scatter(y_t, y_p, s=26, alpha=.7, color="#4C72B0", edgecolor="white")
    lim = [0, max(y_t.max(), y_p.max()) * 1.06]
    ax.plot(lim, lim, "r--", lw=1.5, label="理想预测线 y=x")
    ax.set_xlim(lim); ax.set_ylim(lim)
    ax.set_xlabel("真实租赁量"); ax.set_ylabel("预测租赁量")
    ax.set_title(f"{ttl}\n$R^2$={m['R2']:.3f}, RMSE={m['RMSE']:.1f}, MAE={m['MAE']:.1f}", fontsize=10.5)
    ax.legend(fontsize=8.5, loc="upper left"); ax.grid(alpha=.25, ls="--")
fig.suptitle("预测值 vs 真实值", fontsize=13, y=1.02)
fig.tight_layout(); fig.savefig("figures/fig5_pred_vs_true.png"); plt.close(fig)

# 图6：残差诊断
res_tr, res_te = ytr - pred_tr, yte - pred_te
fig, axes = plt.subplots(2, 2, figsize=(11.2, 8.0))
axes[0,0].scatter(pred_tr, res_tr, s=24, alpha=.7, color="#4C72B0", label="训练集")
axes[0,0].scatter(pred_te, res_te, s=24, alpha=.7, color="#DD8452", marker="^", label="测试集")
axes[0,0].axhline(0, color="k", lw=1)
axes[0,0].set_xlabel("预测值"); axes[0,0].set_ylabel("残差")
axes[0,0].set_title("(a) 残差 vs 预测值", fontsize=11); axes[0,0].legend(fontsize=8.5); axes[0,0].grid(alpha=.25, ls="--")
axes[0,1].hist(res_tr, bins=18, color="#4C72B0", edgecolor="white", alpha=.9)
axes[0,1].axvline(0, color="r", lw=1.2, ls="--")
axes[0,1].set_title("(b) 训练集残差分布", fontsize=11); axes[0,1].grid(alpha=.25, ls="--")
stats.probplot(res_tr, dist="norm", plot=axes[1,0])
axes[1,0].get_lines()[0].set(marker="o", ms=4, alpha=.7, color="#4C72B0")
axes[1,0].get_lines()[1].set(color="r", lw=1.4)
axes[1,0].set_title("(c) 训练集残差 Q-Q 图", fontsize=11); axes[1,0].grid(alpha=.25, ls="--")
axes[1,0].set_xlabel("理论分位数"); axes[1,0].set_ylabel("样本分位数")
axes[1,1].scatter(tr["温度(°C)"], res_tr, s=24, alpha=.7, color="#55A868")
axes[1,1].axhline(0, color="k", lw=1)
xs = np.linspace(tr["温度(°C)"].min(), tr["温度(°C)"].max(), 50)
coef_t = np.polyfit(tr["温度(°C)"], res_tr, 2)
axes[1,1].plot(xs, np.polyval(coef_t, xs), color="#C44E52", lw=2, label="二次拟合趋势")
axes[1,1].set_xlabel("温度(°C)"); axes[1,1].set_ylabel("残差")
axes[1,1].set_title("(d) 残差 vs 温度（暴露线性假设的不足）", fontsize=11)
axes[1,1].legend(fontsize=8.5); axes[1,1].grid(alpha=.25, ls="--")
fig.suptitle("残差诊断", fontsize=13, y=1.0)
```

```
fig.tight_layout(); fig.savefig("figures/fig6_residuals.png"); plt.close(fig)

# 残差正态性 & 异方差检验
sh_w, sh_p = stats.shapiro(res_tr)
bp_stat, bp_p = stats.jarque_bera(res_tr)[0], stats.jarque_bera(res_tr)[1]
R["residual_tests"] = {"shapiro_W": float(sh_w), "shapiro_p": float(sh_p),
                       "jb_stat": float(bp_stat), "jb_p": float(bp_p),
                       "resid_mean": float(res_tr.mean()), "resid_std": float(res_tr.std(ddof=1))}
# 残差与真实值的相关(检测系统性偏差)
R["resid_corr_y"] = float(np.corrcoef(res_tr, ytr)[0, 1])

# ============================================================
# 5. 进阶探索
# ============================================================
adv = []
# (a) 岭回归
for a in [0.1, 1.0, 10.0]:
    rg = Ridge(alpha=a).fit(Ztr.values, ytr)
    adv.append({"name": f"岭回归 alpha={a}",
                "train_R2": metrics(ytr, rg.predict(Ztr.values))["R2"],
                "test_R2": metrics(yte, rg.predict(Zte.values))["R2"],
                "test_RMSE": metrics(yte, rg.predict(Zte.values))["RMSE"]})
# (b) 对数目标 (log(1+y)), 反变换需校正
ylog = np.log1p(ytr)
lr_log = LinearRegression().fit(Ztr.values, ylog)
p_te_log = np.expm1(lr_log.predict(Zte.values))
adv.append({"name": "对数目标 log(1+y)", "train_R2": None,
            "test_R2": metrics(yte, p_te_log)["R2"], "test_RMSE": metrics(yte, p_te_log)["RMSE"]})
# (c) 增加 温度² 与 温度×季节交互
Ztr_x = Ztr.copy(); Zte_x = Zte.copy()
for D in (Ztr_x, Zte_x):
    D["温度²"] = D["温度(°C)"] ** 2
for s in ["春季", "夏季", "秋季"]:
    Ztr_x[f"温度×{s}"] = Ztr_x["温度(°C)"] * Ztr_x[f"季节_{s}"]
    Zte_x[f"温度×{s}"] = Zte_x["温度(°C)"] * Zte_x[f"季节_{s}"]
lr_x = LinearRegression().fit(Ztr_x.values, ytr)
adv.append({"name": "线性回归 + 温度² + 温度×季节交互",
            "train_R2": metrics(ytr, lr_x.predict(Ztr_x.values))["R2"],
            "test_R2": metrics(yte, lr_x.predict(Zte_x.values))["R2"],
            "test_RMSE": metrics(yte, lr_x.predict(Zte_x.values))["RMSE"]})
# (b2) 对数目标 + Duan 平滑(涂抹)校正
res_log = ylog - lr_log.predict(Ztr.values)
smear = float(np.mean(np.exp(res_log)))
p_te_duan = np.expm1(lr_log.predict(Zte.values) + np.log(smear))
adv.append({"name": "对数目标 log(1+y) + Duan校正", "train_R2": None,
            "test_R2": metrics(yte, p_te_duan)["R2"], "test_RMSE": metrics(yte, p_te_duan)["RMSE"]})
R["duan_smear"] = smear
R["advanced"] = adv

# 方差膨胀因子 VIF（共线性诊断）
vif = {}
D = Xtr_raw.values.astype(float)
for j, c in enumerate(feat_names):
    others = np.delete(D, j, axis=1)
    r2j = LinearRegression().fit(others, D[:, j]).score(others, D[:, j])
    vif[c] = float(1.0 / max(1e-12, 1 - r2j))
R["vif"] = vif

# 预测值分布 vs 真实值分布（观察“均值回归”压缩效应）
R["pred_stats"] = {"true_std": float(ytr.std(ddof=1)), "pred_std_train": float(pred_tr.std(ddof=1)),
                   "true_range": [float(ytr.min()), float(ytr.max())],
                   "pred_range_train": [float(pred_tr.min()), float(pred_tr.max())],
                   "corr_true_pred_train": float(np.corrcoef(ytr, pred_tr)[0, 1]),
                   "corr_true_pred_test": float(np.corrcoef(yte, pred_te)[0, 1]),
                   "pred_negative_count_train": int((pred_tr < 0).sum()),
                   "pred_negative_count_test": int((pred_te < 0).sum())}
# 分区间误差（检查误差是否随需求水平增大）
bins_y = [0, 300, 800, 1500, np.inf]
seg = []
for lo, hi in zip(bins_y[:-1], bins_y[1:]):
    m = (yte >= lo) & (yte < hi)
    if m.sum() > 0:
        seg.append({"range": f"[{int(lo)}, {int(hi) if np.isfinite(hi) else '+'})", "n": int(m.sum()),
                    "MAE": float(np.mean(np.abs(res_te[m]))), "bias": float(np.mean(res_te[m])),
                    "RMSE": float(np.sqrt(np.mean(res_te[m] ** 2)))})
R["segment_test"] = seg

# ============================================================
# 6. 导出预测结果与结果 JSON
# ============================================================
out_tr = tr.copy(); out_tr["预测值"] = np.round(pred_tr, 1); out_tr["残差"] = np.round(res_tr, 1)
out_te = te.copy(); out_te["预测值"] = np.round(pred_te, 1); out_te["残差"] = np.round(res_te, 1)
out_tr.to_csv("results/训练集_预测与残差.csv", index=False, encoding="utf-8-sig")
out_te.to_csv("results/测试集_预测与残差.csv", index=False, encoding="utf-8-sig")
```

```
with open("results/results.json", "w", encoding="utf-8") as f:
    json.dump(R, f, ensure_ascii=False, indent=1)

print("=" * 66)
print("求解一致性: 正规方程 vs sklearn 最大差 =", f"{R['solver_agreement']['max_diff_ne_sk']:.2e}",
      "| vs 梯度下降 =", f"{R['solver_agreement']['max_diff_ne_gd']:.2e}")
print("参数反变换校验 |Δpred|max =", f"{R['param_invariance_maxdiff']:.2e}")
print(f"训练集: R2={m_tr['R2']:.4f}  RMSE={m_tr['RMSE']:.2f}  MAE={m_tr['MAE']:.2f}  MAPE={m_tr['MAPE']:.2f}%")
print(f"测试集: R2={m_te['R2']:.4f}  RMSE={m_te['RMSE']:.2f}  MAE={m_te['MAE']:.2f}  MAPE={m_te['MAPE']:.2f}%")
print("原始尺度参数:", {k: round(v, 3) for k, v in R["params_raw"].items()})
print("5折CV R2:", [round(v, 3) for v in R["cv"]["R2_folds"]], "均值", round(R["cv"]["R2_mean"], 3))
print("VIF:", {k: round(v, 2) for k, v in R["vif"].items()})
print("预测区间(训练):", R["pred_stats"]["pred_range_train"], "真实区间:", R["pred_stats"]["true_range"],
      "| pred std/truestd =", round(R["pred_stats"]["pred_std_train"] / R["pred_stats"]["true_std"], 3))
print("分段误差(测试集):", R["segment_test"])
print("=" * 66)
print("图: ", sorted(os.listdir("figures")))
```

附录 B 交付文件清单与复现说明

本次作业交付以下文件（与 PDF 报告一同提交）：

#	文件	说明
1	姓名-学号-共享单车回归作业.pdf	本报告（提交给助教的唯一文件，发送至 liushuaiyu@stu.xjtu.edu.cn）
2	共享单车_线性回归_分析.py	完整可复现代码（数据读取→建模→评估→绘图→结果导出）
3	figures/（7 张图 + 公式图）	报告全部插图，300+ dpi
4	results/results.json	全部数值结果（供复核）
5	results/训练集_预测与残差.csv、测试集_预测与残差.csv	逐样本预测值、残差明细
6	上传数据（训练/测试 CSV）	作业原始数据，未做任何修改

复现步骤：将两个数据 CSV 置于 uploads/ 目录后，在项目根目录执行 python3 共享单车_线性回归_分析.py 即可重新生成全部图表与 results.json；本报告 PDF 由脚本读取 results.json 后编译生成，全部随机性仅出现在 5 折交叉验证的划分中，已固定随机种子（seed = 42），结果完全可复现。

数据来源：UCI Machine Learning Repository, Seoul Bike Sharing Demand —— https://archive.ics.uci.edu/dataset/560/seoul+bike+sharing+demand（作业提供其子集）。